Parnassus

Neural Fast Simulation and Reconstruction for High Energy Physics

Product Reference Document

Version 0.1.0

Etienne Dreyer[1], Eilam Gross[1], Dmitrii Kobylianskii[1], Vinicius Mikuni[2], Benjamin Nachman[2]

[1]Weizmann Institute of Science, Rehovot, Israel

[2]Lawrence Berkeley National Laboratory, Berkeley, CA, USA

March 13, 2026

Contents

1 Introduction

Parnassus is a **pure-Python**, **fully GPU-compatible** framework for **fast detector simulation and reconstruction** in High Energy Physics (HEP). It replaces the computationally expensive full GEANT4 [3] simulation and reconstruction chain with neural network-based generative models that learn the mapping from truth-level (generator-level) particles directly to detector-level particle-flow [4, 5] (pflow) objects, bypassing intermediate steps such as hit simulation, digitisation, and track reconstruction. The framework is both **process-agnostic** and **detector-agnostic**: the same architecture supports multiple detector models, and for each detector the pre-trained model can be applied to arbitrary physics processes by simply providing different input truth events.

Unlike traditional fast simulation tools such as DELPHES [25], which are written in C++, require the ROOT framework [19], and run only on CPU, Parnassus is implemented entirely in Python with PyTorch [22] and runs natively on GPU (CUDA and Apple MPS), enabling seamless integration into modern Python-based analysis workflows and significant acceleration on GPU hardware. ROOT I/O is supported via `uproot` [24] for writing output files, but is entirely optional — Parnassus has no dependency on ROOT itself.

Users select a **detector model** — analogous to choosing a detector card in DELPHES — to target a specific experiment. The current release ships with a CMS detector model trained on 2011 open data [6]; additional detector models for other experiments will be added in future releases.

The physics methodology underlying Parnassus has been described in detail in Refs. [1, 2], where we demonstrated that the approach reproduces full CMS simulation and reconstruction across a wide range of Standard Model and BSM processes. In this document we focus on the **public software release** — its installation, usage, and API — to enable the broader HEP community to adopt neural fast simulation and reconstruction in their workflows.

The neural backbone uses conditional flow matching [7, 8] to train continuous-time diffusion models [9, 10] that are sampled via Euler integration at inference time. The models are conditioned on truth-level particle properties, making the generation inherently process-agnostic without requiring retraining for new physics scenarios.

1.1 Design Goals

Speed Orders of magnitude faster than full simulation and reconstruction by replacing the detector response with neural diffusion models. Fully GPU-compatible for additional acceleration.

Pure Python Implemented entirely in Python with no compiled dependencies beyond PyTorch. Unlike C++/ROOT-based tools such as DELPHES [25], Parnassus integrates naturally into modern Python analysis ecosystems. ROOT output is available via `uproot` but is not required.

Fidelity Detector models are trained on full simulation and reconstruction data to reproduce realistic detector-level distributions including particle kinematics, vertex positions, impact parameters, and particle identification. The current CMS detector model, trained on 2011 open data [6], shows excellent agreement with full CMS simulation across diverse processes [1, 2].

Flexibility Accept truth-level input from any source — PYTHIA8 [11] on-the-fly generation, HepMC [12] files from external generators (MADGRAPH5_aMC@NLO [13], SHERPA [14], HERWIG [15], POWHEG [16]), or custom formats. Users select a

detector model to target a specific experiment, analogous to choosing a detector card in DELPHES.

Completeness
Full post-processing pipeline including jet clustering (FASTJET [17]) and lepton isolation, with optional output to ROOT format via `uproot` [24].

1.2 What Parnassus Produces

Given a set of truth-level particles for each event, Parnassus generates the quantities listed in Table 1.

Table 1: Output quantities produced by Parnassus for each event.

Output	Description
Particle-flow particles	Full kinematics (p_T, η, ϕ), vertex (v_x, v_y, v_z), class, PDG ID
Impact parameters	d_0, z_0 and their errors for charged particles
Jets	Clustered with configurable algorithms (anti-k_t [18], gen-k_t); substructure (D_2 [20], C
Lepton isolation	Isolation scores and p_T sums in configurable ΔR cones
Event-level quantities	H_T, $\vec{E}_\mathrm{T}^{\mathrm{miss}}$ components, particle multiplicity

2 Architecture Overview

Parnassus uses a three-stage neural generation pipeline followed by physics-based post-processing. The architecture is illustrated in Figure 1.

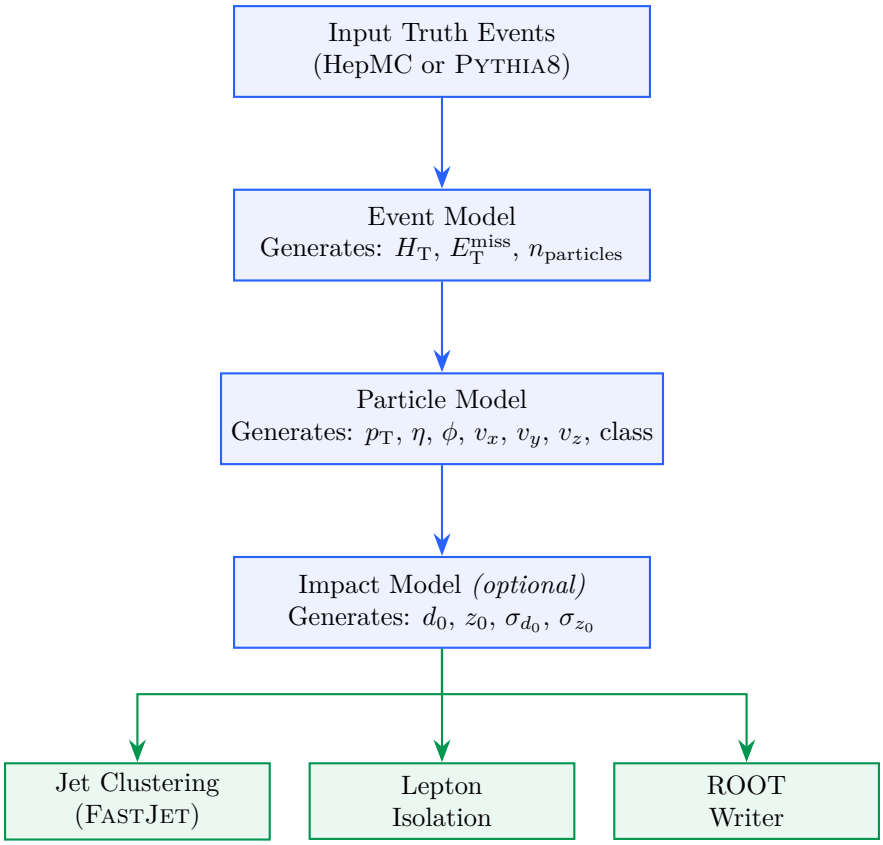

Figure 1: Parnassus generation pipeline. Three neural diffusion models (blue) produce detector-level quantities from truth-level input. Physics-based post-processing stages (green) compute derived quantities and write the final output.

All three neural models are **diffusion models** trained with conditional flow matching [7, 8] and sampled via Euler integration. Each model is conditioned on truth-level particle information, making the generation process inherently dependent on the input physics process without being tied to any specific one.

2.1 Neural Model Details

Table 2 summarises the three neural models and their input/output specifications.

Table 2: Neural model specifications (shown for the CMS 2011 detector model). The same architecture applies to all detector models.

Model	Input Context	Output Variables	Sampler
Event	Truth particle kinematics (p_T^{rel}, η, ϕ, v_x, v_y, v_z, class), global sums (H_T, MET, n_{truth})	pflow H_T, MET_x, MET_y, n_{pflow}	Euler, 50 steps
Particle	Same + event model outputs	pflow p_T^{rel}, η, ϕ, v_x, v_y, v_z, class	Euler (reverse time), 50 steps
Impact	Same + event model outputs	d_0, z_0, σ_{d_0}, σ_{z_0}	Euler, 50 steps

The number of diffusion steps is configurable at runtime via the `--num_steps` (`-n`) flag, allowing users to trade generation speed for quality.

3 Installation and Quick Start

3.1 Installation

```
# Clone the repository
git clone https://github.com/parnassus-hep/parnassus.git
cd parnassus

# Install with pip (requires Python 3.12)
pip install .

# For development
pip install -e ".[dev]"
```

Listing 1: Installation commands.

3.2 Requirements

- Python ≥ 3.12, < 3.13
- PyTorch [22] $\geq 2.6.0$
- Pythia8mc [11] 8.316.0
- PyHepMC [12] $\geq 2.14.0$
- FastJet [17] $\geq 3.4.3.1$
- EnergyFlow [23] $\geq 1.4.0$
- Uproot [24] $\geq 5.5.2$ *(optional, for ROOT output)*

3.3 Initialise a Configuration File

```
parnassus init .
```

This copies the default `default_config.yaml` to the current directory. The default configuration uses the `cms_2011_flow_v00` detector model with anti-k_t jet clustering and lepton isolation. To target a different detector, change the `generator.name` field to the desired detector model.

3.4 Run with a HepMC Input File

```
parnassus run \
  -c default_config.yaml \
  -i events.hepmc \
  -ne 1000 \
  -bs 2000 \
  -o output.root
```

Listing 2: Basic command-line invocation.

The available command-line flags are listed in Table 3.

Table 3: Command-line flags for `parnassus run`.

Flag	Long form	Description
`-c`	`--config`	Path to YAML configuration file
`-i`	`--input_path`	Input HepMC or Pythia `.cmnd` file
`-o`	`--output_path`	Output ROOT file path
`-ne`	`--num_events`	Number of events to process
`-bs`	`--batch_size`	Batch size for neural generation
`-n`	`--num_steps`	Diffusion sampler steps (default: 50)

3.5 Run with Pythia8 On-the-Fly Generation

To generate truth-level events on-the-fly with PYTHIA8, create a `.cmnd` steering card and pass it as the input file. Parnassus detects `.cmnd` files automatically:

```
parnassus run \
  -c default_config.yaml \
  -i ttbar.cmnd \
  -ne 5000 \
  -bs 2000 \
  -o ttbar_fastsim.root
```

4 Demonstration: SM and BSM Processes

The neural models are **process-agnostic** — they learn the detector response as a function of truth-level particle properties, not the physics process itself. We have previously demonstrated [1, 2] that Parnassus reproduces full simulation and reconstruction across a wide range of SM and BSM processes. Here we show two representative examples using the CMS detector model to illustrate the API in action: a Standard Model process ($H \to ZZ \to 4\ell$) and a BSM exotic Higgs decay ($H \to aa \to gg\mu\mu$). The same workflow applies to any detector model by changing the `generator.name` in the configuration.

The current CMS detector model supports up to 400 truth particles per event. For events exceeding this limit, Parnassus retains the 400 highest-p_{T} particles so that no events are discarded.

4.1 Standard Model: $H \to ZZ \to 4\ell$

We process 100 events from a HepMC input file produced by PYTHIA8 [11]:

```
parnassus run \
  -c default_config.yaml \
  -i h4l_events.hepmc \
  -ne 100 \
  -o h4l_output.root
```

4.2 BSM: $H \to aa \to gg\mu\mu$

For the BSM example, we generate exotic Higgs decays on-the-fly with PYTHIA8. The Higgs decays to a pair of light pseudoscalars a ($m_a = 20\,\mathrm{GeV}$), each decaying to either gg or $\mu^+\mu^-$, producing a mixed final state of jets and muon pairs:

```
! haa_ggmumu.cmnd -- BSM exotic Higgs decay
Beams:eCM = 13000.
Beams:idA = 2212
Beams:idB = 2212

! BSM Higgs production via gluon fusion
Higgs:useBSM = on
HiggsBSM:gg2H2 = on
35:m0 = 125.0                          ! Higgs mass

! SM-like couplings for production
HiggsH2:coup2u = 1.0
HiggsH2:coup2d = 1.0
HiggsH2:coup2A3A3 = 1.0                ! H -> aa coupling

! Force H -> a a only
35:onMode = off
35:onIfAll = 36 36

! Light pseudoscalar a properties (PDG ID 36)
36:m0 = 20.0                           ! mass = 20 GeV
HiggsA3:coup2d = 1.0
HiggsA3:coup2u = 1.0
HiggsA3:coup2l = 1.0

! Force a -> gg or mumu only
36:onMode = off
36:onIfAny = 21 13
```

Listing 3: Pythia8 steering card for BSM $H \to aa \to gg\mu\mu$.

```
parnassus run \
  -c default_config.yaml \
  -i haa_ggmumu.cmnd \
  -ne 1000 \
  -o haa_ggmumu_output.root
```

This demonstrates running Parnassus on a BSM signal process not present in the training data. The mixed $gg\mu\mu$ final state probes the model's ability to simultaneously generate hadronic jets and isolated muons within the same event.

4.3 Kinematic Distributions

Figure 2 compares the particle-flow kinematic distributions for both processes. The p_{T} spectra follow the expected steeply falling shape, the η distributions reflect the detector acceptance, and the multiplicity distributions reflect the different underlying event structures. The BSM $H \to aa$ process shows higher multiplicity due to the hadronic activity from the $a \to gg$ decays.

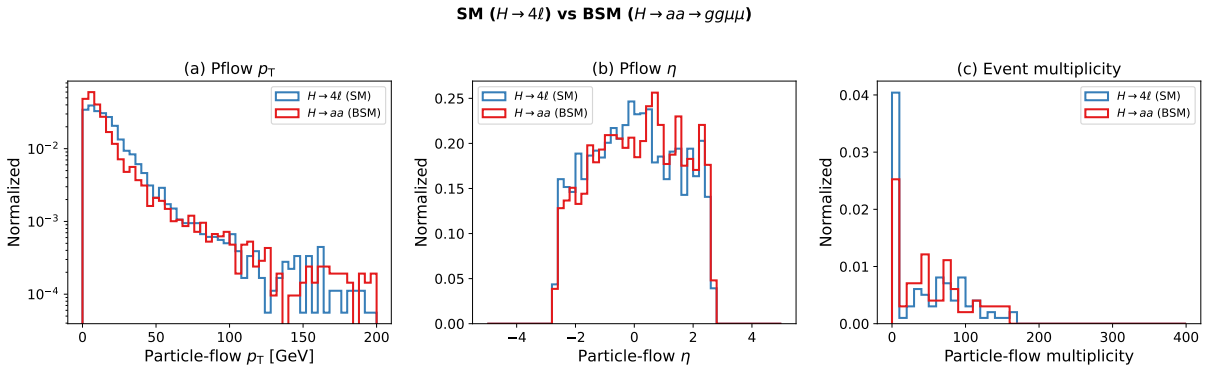

Figure 2: Particle-flow kinematic distributions comparing SM ($H \to 4\ell$, 99 events) and BSM ($H \to aa \to gg\mu\mu$, 99 events) processes. (a) p_T in log scale, (b) pseudorapidity η, (c) particle-flow multiplicity per event. All distributions are normalised to unit area.

4.4 Event Display

Figure 3 shows a representative $H \to 4\ell$ event in the η–ϕ plane, illustrating how Parnassus transforms truth-level particles into detector-level pflow objects.

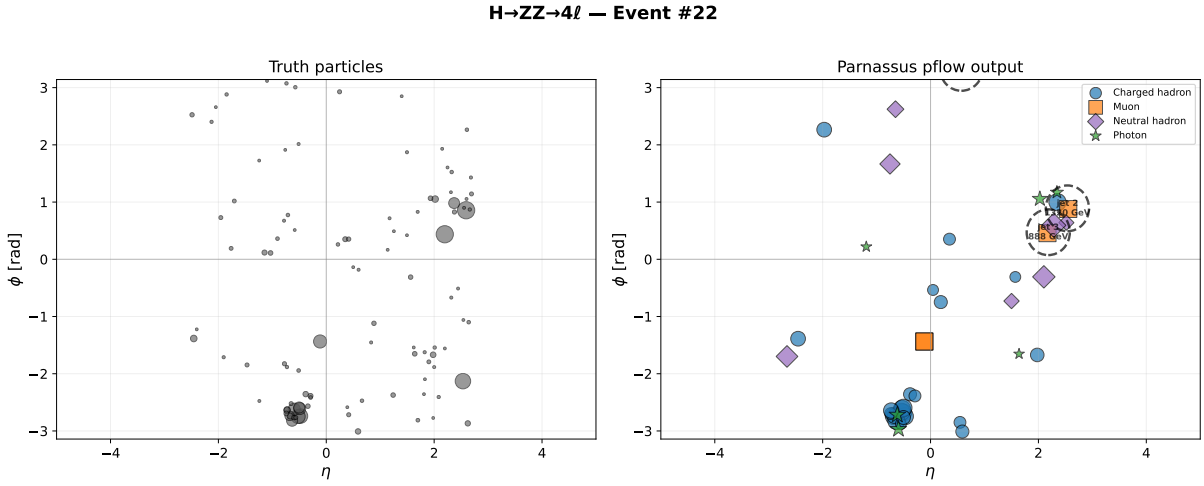

Figure 3: Event display for $H \to ZZ \to 4\ell$: truth particles (left) and Parnassus pflow output (right) in the η–ϕ plane. Marker size scales with p_T; dashed circles indicate anti-k_t jets ($R = 0.5$). Particle classes: charged hadrons (blue circles), electrons (red triangles), muons (orange squares), neutral hadrons (purple diamonds), photons (green stars).

4.5 Aggregate Statistics

Table 4 compares the aggregate statistics for both processes. The BSM process shows higher truth-particle multiplicity (163 ± 70) due to the $a \to gg$ hadronic decays, while the pflow output multiplicity is comparable across both processes.

Table 4: Aggregate statistics for SM and BSM processes (mean ± std).

Quantity	$H \to 4\ell$ **(99 evt)**	$H \to aa$ **(99 evt)**
Truth particles/event	97.2 ± 46.0	163.4 ± 69.9
Pflow particles/event	46.3 ± 47.2	53.6 ± 46.4

4.6 Particle Class Distribution

Figure 4 compares the particle class composition between the two processes. Both show similar class fractions dominated by charged hadrons (∼60%), consistent with the process-agnostic nature of the model.

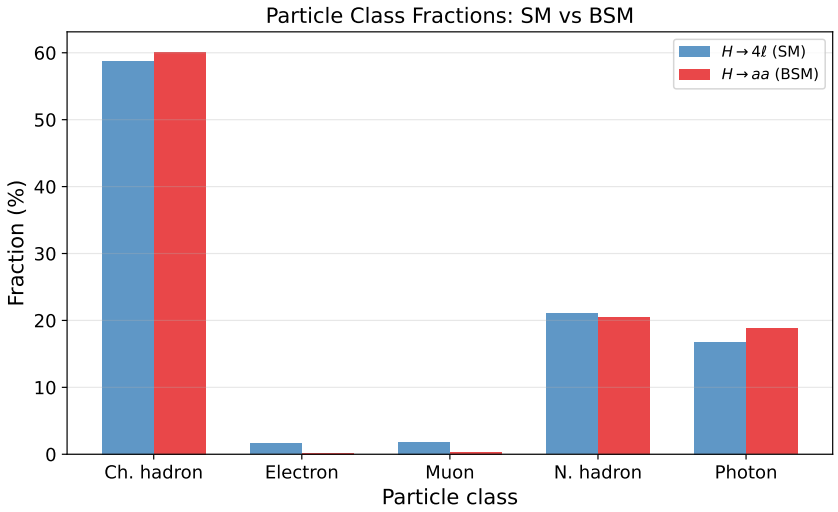

Figure 4: Particle class fractions for SM ($H \to 4\ell$) and BSM ($H \to aa \to gg\mu\mu$) processes. The model produces consistent class compositions regardless of the input physics process.

5 Diffusion Steps Convergence

The number of diffusion steps N controls the trade-off between generation quality and speed. Figure 5 shows the convergence of particle-flow kinematic distributions as a function of N for the $H \to 4\ell$ sample.

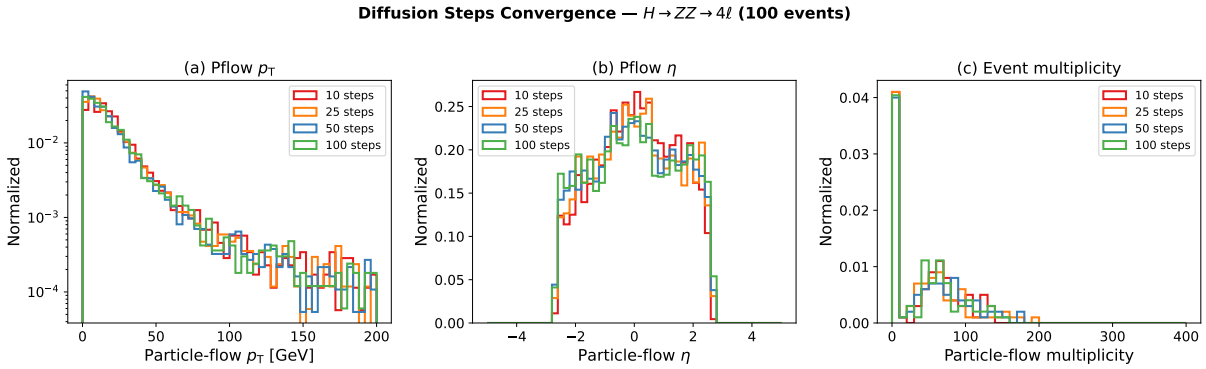

Figure 5: Diffusion steps convergence for $H \to ZZ \to 4\ell$ (100 events). (a) Pflow p_T, (b) pflow η, (c) event multiplicity. Distributions are stable from $N = 25$ onward.

Figure 6 shows that particle class fractions are also stable across step counts.

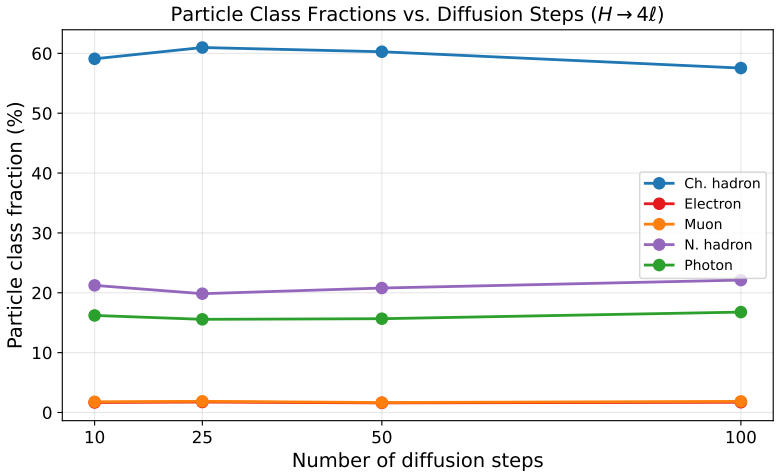

Figure 6: Particle class fractions vs. number of diffusion steps. All classes converge by $N = 25$.

Table 5 summarises the generation time as a function of diffusion steps on CPU.

Table 5: Generation time vs. diffusion steps (100 events, CPU).

Steps	Wall time
10	∼1:30
25	∼1:30
50	∼2:10
100	∼5:20

The default of 50 steps provides a good balance of quality and speed. For quick validation runs, 25 steps yields nearly identical distributions at lower cost.

6 Python API

For programmatic usage, Parnassus provides a clean Python API:

```python
from pathlib import Path
from parnassus.configs import Config
from parnassus.pipelines import (
    GenerationPipeline,
    JetClusteringPipeline,
    IsolationPipeline,
)
from parnassus.configs.pipeline import (
    JetClusteringConfig,
    IsolationConfig,
)
from parnassus.writers import RootWriter

# Load configuration
config = Config.from_yaml("my_config.yaml")

# Override settings programmatically
config.dataset_config.file_path = (
    Path("my_events.hepmc").absolute()
)
```

```
config.dataset_config.num_events = 5000

# Run generation
pipeline = GenerationPipeline(config)
gen_events, accessors_dict = pipeline.run()

# Post-processing
for pipeline_config in config.pipeline_configs:
    if isinstance(pipeline_config, JetClusteringConfig):
        jet_pipeline = JetClusteringPipeline(pipeline_config)
        jet_pipeline.process(gen_events)
    elif isinstance(pipeline_config, IsolationConfig):
        iso_pipeline = IsolationPipeline(pipeline_config)
        iso_pipeline.process(gen_events)

# Inspect generated data
for event in gen_events[:5]:
    print(f"Event {event.event_number}:")
    n_truth = len(event.truth_particles.pt)
    n_pflow = len(event.pflow_particles.pt)
    print(f"  Truth particles: {n_truth}")
    print(f"  Pflow particles: {n_pflow}")
    print(f"  HT = {event.ht:.1f} GeV")

# Write to ROOT
writer = RootWriter(config.writer_config)
writer.write(gen_events)
```

Listing 4: Programmatic usage of the generation pipeline.

6.1 Reading Output Files

```
import uproot
import numpy as np

f = uproot.open("output.root")
tree = f["Parnassus"]

# Access pflow jet pT
jet_pt = tree["PflowJetsAntiKt05.pt"].array()

# Access electron isolation
e_iso = tree["Electrons.iso_var"].array()

# Access impact parameters
d0 = tree["Pflow.d0"].array()
z0 = tree["Pflow.z0"].array()
```

Listing 5: Reading the output ROOT file with `uproot`.

7 Configuration Reference

```
# -- Dataset -----------------------------------------
dataset:
```

```yaml
  file_path: ""               # Path to .hepmc or .cmnd file
  num_events: 1000            # Number of events to process
  entry_start: 0              # Starting event index (HepMC)

# -- Generator ---------------------------------------
generator:
  type: "neural"              # "neural" or "parametric"
  name: "cms_2011_flow_v00"   # Detector model (like a Delphes card)
  num_steps: 50               # Diffusion steps (neural only)
  batch_size: 2000            # Events per batch
  device: "cpu"               # "cpu", "cuda", or "mps"

# -- Post-processing Pipelines ------------------------
pipelines:
  TruthJetsAntiKt05:
    type: "cluster"
    collection: truth
    dr: 0.5
    algorithm: antikt
    pt_min: 10
    nconst_min: 2
  PflowJetsAntiKt05:
    type: "cluster"
    collection: pflow
    dr: 0.5
    algorithm: antikt
    pt_min: 10
    nconst_min: 2
  ElectronIsolation:
    type: "isolation"
    collection: "electrons"
    dr: 0.4
  MuonIsolation:
    type: "isolation"
    collection: "muons"
    dr: 0.4

# -- Output ------------------------------------------
output:
  file_path: ""               # Output ROOT file path
  format: default
```

Listing 6: Annotated default configuration.

8 Conclusion

We have presented the public release of Parnassus, a pure-Python, GPU-compatible framework for neural fast simulation and reconstruction in High Energy Physics. Unlike traditional C++/ROOT-based tools such as DELPHES [25], Parnassus runs entirely in Python with PyTorch, supports GPU acceleration (CUDA and Apple MPS), and requires no ROOT installation — ROOT output is optionally handled via `uproot`. The software provides a simple command-line and Python API for generating detector-level particle-flow objects from truth-level input, with full post-processing including jet clustering and lepton isolation.

The framework is both process-agnostic and detector-agnostic. Users select a detector model — analogous to choosing a detector card in DELPHES — to target a specific experiment. The current release ships with a CMS detector model validated against full CMS simulation across

diverse SM and BSM processes [1, 2]; additional detector models will be provided in future releases.

The source code is publicly available at https://github.com/parnassus-hep/parnassus under the MIT license.

References

[1] E. Dreyer, E. Gross, D. Kobylianskii, V. Mikuni, B. Nachman, and N. Soybelman, "Parnassus: An Automated Approach to Accurate, Precise, and Fast Detector Simulation and Reconstruction," arXiv:2406.01620 (2024).

[2] E. Dreyer, E. Gross, D. Kobylianskii, V. Mikuni, and B. Nachman, "Conditional Deep Generative Models for Simultaneous Simulation and Reconstruction of Entire Events," arXiv:2503.19981 (2025).

[3] S. Agostinelli *et al.* (GEANT4 Collaboration), "Geant4 — a simulation toolkit," *Nucl. Instrum. Meth. A* **506**, 250 (2003).

[4] CMS Collaboration, "Particle-flow reconstruction and global event description with the CMS detector," *JINST* **12**, P10003 (2017), arXiv:1706.04965.

[5] ATLAS Collaboration, "Jet reconstruction and performance using particle flow with the ATLAS Detector," *Eur. Phys. J. C* **77**, 466 (2017), arXiv:1703.10485.

[6] CMS Collaboration, "CMS Open Data," https://opendata.cern.ch/search?experiment=CMS.

[7] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, "Flow Matching for Generative Modeling," *ICLR* (2023), arXiv:2210.02747.

[8] X. Liu, C. Gong, Q. Liu, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow," *ICLR* (2023), arXiv:2209.03003.

[9] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, B. Poole, "Score-Based Generative Modeling through Stochastic Differential Equations," *ICLR* (2021), arXiv:2011.13456.

[10] J. Ho, A. Jain, P. Abbeel, "Denoising Diffusion Probabilistic Models," *NeurIPS* (2020), arXiv:2006.11239.

[11] T. Sjöstrand *et al.*, "An introduction to PYTHIA 8.2," *Comput. Phys. Commun.* **191**, 159 (2015), arXiv:1410.3012.

[12] A. Buckley *et al.*, "The HepMC3 event record library for Monte Carlo event generators," *Comput. Phys. Commun.* **260**, 107310 (2021), arXiv:1912.08005.

[13] J. Alwall *et al.*, "The automated computation of tree-level and next-to-leading order differential cross sections, and their matching to parton shower simulations," *JHEP* **07**, 079 (2014), arXiv:1405.0301.

[14] E. Bothmann *et al.* (SHERPA Collaboration), "Event Generation with Sherpa 2.2," *SciPost Phys.* **7**, 034 (2019), arXiv:1905.09127.

[15] J. Bellm *et al.*, "Herwig 7.0/Herwig++ 3.0 release note," *Eur. Phys. J. C* **76**, 196 (2016), arXiv:1512.01178.

[16] S. Alioli, P. Nason, C. Oleari, E. Re, "A general framework for implementing NLO calculations in shower Monte Carlo programs: the POWHEG BOX," *JHEP* **06**, 043 (2010), arXiv:1002.2581.

[17] M. Cacciari, G. P. Salam, G. Soyez, "FastJet User Manual," *Eur. Phys. J. C* **72**, 1896 (2012), arXiv:1111.6097.

[18] M. Cacciari, G. P. Salam, G. Soyez, "The anti-k_t jet clustering algorithm," *JHEP* **04**, 063 (2008), arXiv:0802.1189.

[19] R. Brun and F. Rademakers, "ROOT — An object oriented data analysis framework," *Nucl. Instrum. Meth. A* **389**, 81 (1997).

[20] A. J. Larkoski, I. Moult, D. Neill, "Power Counting to Better Jet Observables," *JHEP* **12**, 009 (2014), arXiv:1409.6298.

[21] A. J. Larkoski, G. P. Salam, J. Thaler, "Energy Correlation Functions for Jet Substructure," *JHEP* **06**, 108 (2013), arXiv:1305.0007.

[22] A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," *NeurIPS* (2019), arXiv:1912.01703.

[23] P. T. Komiske, E. M. Metodiev, J. Thaler, "EnergyFlow: A Python framework for jet and event analysis," (2019).

[24] J. Pivarski *et al.*, "Uproot: ROOT I/O in pure Python and NumPy," `https://github.com/scikit-hep/uproot5`.

[25] J. de Favereau *et al.* (DELPHES 3 Collaboration), "DELPHES 3: A modular framework for fast simulation of a generic collider experiment," *JHEP* **02**, 057 (2014), arXiv:1307.6346.

[26] A. Butter *et al.*, "Machine Learning and LHC Event Generation," *SciPost Phys.* **14**, 079 (2023), arXiv:2203.07460.

[27] C. Krause *et al.*, "CaloChallenge: Community comparison of fast calorimeter simulation," (2024), arXiv:2410.21611.